

Automated Hit-frame Detection for Badminton Match Analysis

Yu-Hang Chien, Fang Yu

National Chengchi University, Taipei, Taiwan
moneychien20639@gmail.com, yuf@nccu.edu.tw

Abstract

Sports professionals constantly under pressure to perform at the highest level can benefit from sports analysis, which allows coaches and players to reduce manual efforts and systematically evaluate their performance using automated tools. This research aims to advance sports analysis in badminton, systematically detecting hit-frames automatically from match videos using modern deep learning techniques. The data included in hit-frames can subsequently be utilized to synthesize players' strokes and on-court movement, as well as for other downstream applications such as analyzing training tasks and competition strategy. The proposed approach in this study comprises several automated procedures like rally-wise video trimming, player and court keypoints detection, shuttlecock flying direction prediction, and hit-frame detection. In the study, we achieved 99% accuracy on shot angle recognition for video trimming, over 92% accuracy for applying player keypoints sequences on shuttlecock flying direction prediction, and reported the evaluation results of rally-wise video trimming and hit-frame detection.

Introduction

Badminton, the fastest racquet sport in the world, is played by players of all ages and can be played in singles or doubles configurations (Phomsoupha and Laffaye 2015; Ramasamy et al. 2021). Badminton is also a fast-paced and explosive sport that requires athletes' limbs to perform sudden motions such as arm and racquet movements, jumps, lunges, and quick direction changes to deliver various types of strokes, (Rambely and Osman 2005; Kuntze, Mansfield, and Sellers 2010; Huang et al. 2016). By 2022, badminton is expected to have 220 million active players, according to the Badminton World Federation (BWF). As the sport becomes increasingly popular, there are a rising number of research and analysis on it. Stroke and on-court movement sequences are commonly used in badminton sports analysis to show how the match progressed and offer tactical information; hit-frames, or the frames when a player strikes the shuttlecock and alters its flight direction, can be utilized to obtain this information. Additionally, hit-frames possess essential data, including the players' striking location, the shuttlecock's landing zone, and their movements as they return the shuttlecock.

This information can be used for further tactical analysis. As a result, such information benefits coaches and players as it aids in improving performance and formulating winning strategies. However, no automated badminton programs can identify hit-frames in raw match videos. By outlining a methodical strategy for automatically identifying hit-frame sequences from publicly available match videos, this work intends to enhance badminton sports analysis. Our method can facilitate downstream applications dependent on information included in hit-frames, such as stroke classification (Chu and Situmeang 2017), movement recognition (Valdecabres et al. 2020), and tactical analysis (Wang 2022).

Related Works

Focusing on trajectory-related studies, (Liu and Wang 2022) reconstructs the 3D trajectory of shots from unlabeled videos using various neural networks and a per-shot trajectory reconstruction method. (Lee 2016) demonstrated a model-based trajectory estimation method based on linear regression and SVM. (Sun et al. 2020) developed Track-NetV2, to forecast the trajectory of the tiny and fast-moving shuttlecock from badminton videos.

For stroke recognition and prediction research, (Wang, Guo, and Zhao 2016) proposed a badminton stroke recognition system based on body sensors and a two-layer hidden Markov model. (Chen and Wang 2007) described applying 2-D seriate images to obtain statistical data from a badminton match. (Chu and Situmeang 2017) integrated visual analysis techniques to detect the court and players, classify strokes, and the player's strategy. (Ghosh, Singh, and Jawahar 2018) proposed a method to analyze badminton broadcast videos by segmenting the points played, tracking and recognizing the players in the points, and commenting on their respective badminton strokes. (Careelmont 2013) attempted to solve the problem of badminton stroke classification in compressed videos using shuttlecock trajectory. (Wang et al. 2022b) propose a novel Position-aware Fusion of Rally Progress and Player Styles framework that incorporates rally progress and information of the players by two modified encoder-decoder extractors. (Wang 2022) offered a unified badminton language to describe the shot process and measured the win probability of each shot in badminton matches by considering long-term and short-term dependencies. (Wang 2022) also introduced a framework with two

encoder-decoder extractors and a position-aware fusion network to forecast the possible tactics of players.

As for studies related to footwork and movements, (Valldcabres et al. 2019) designed a reliable tool for recording and analyzing the behavior of badminton singles players, then took advantage of the tool and conducted a study that examined the relationship between the elite badminton players’ movements and contextual variables (game, round, and match status) (Valldcabres et al. 2020). (Abdullahi and Coetzee 2017) investigated the relationships between singles match strokes and foot movements in African Badminton Championship male badminton players.

These works primarily concentrated on specific components such as trajectory construction, stroke classification, and movement analysis. Moreover, many of them rely on the information within the hit-frames (discussed in), like players’ location and the players’ motions when they hit the shuttlecock, for further analysis and application. To our knowledge, no previous work predicts the shuttlecock’s flying direction with a transformer model, given the keypoints sequence of on-court players. This study provides a systematic way to generate shuttlecock flying direction sequences, then detect hit-frames according to the sequence. Such an approach can facilitate data labeling and downstream applications requiring hit-frames information.

Proposed Approaches

Badminton is a game with a 21-point scoring system. Consequently, it is challenging to collect data from badminton videos all at once. In such a case, we use heuristic methods to divide raw match video into frames based on rallies, forecast the shuttlecock’s direction of flight in each frame, and pinpoint hit-frames. Algorithm 1 depicts the process of all the procedures. The hit-frame generating algorithm inputs a badminton frame sequence (video) V and produces a set of hit-frame indices denoted by Hit . Lines 1 to 3 define the shot angle constants O (representing *other*) and H (representing *high*). These constants are used to realize the rally-wise video trimming module (described in). The variable $previous_shot_angle$ is initialized to O to keep track of the shot angle of the previous frame. Two sets are also created: Kps (for storing detected player keypoints) and Hit (for storing detected hit-frame indices), both initially empty. From line 4, the algorithm proceeds to iterate through each frame $frame$ in the input V . For each frame, the shot angle is recognized using the function `AngleRecognition` (described in), and the algorithm follows a series of conditional checks to identify consecutive frames belonging to the same rally.

1. If the angle of the current frame $angle$ matches the angle of the previous frame $previous_shot_angle$ and equals to H (like line 6), which indicates the rally is still ongoing, the algorithm detects the keypoints kps of such frames using `DetectPlayerKeypoints` (elaborated in) and add kps to the Kps set.
2. If $angle$ matches $previous_shot_angle$ and equals O (like line 10), the algorithm will overlook such frame and iterate back to line 4.

Algorithm 1: Hit-frame generating algorithm.

Input: Badminton frame sequence V .

Output: Hit-frame set Hit .

```

1: Let const  $O \leftarrow 0, H \leftarrow 1$ 
2: Let  $previous\_shot\_angle \leftarrow O$ 
3: Let  $Kps \leftarrow \emptyset, Hit \leftarrow \emptyset$ 
4: for all  $frame \in V$  do
5:    $angle \leftarrow \text{AngleRecognition}(frame)$ 
6:   if  $angle = previous\_shot\_angle$  then
7:     if  $shot\_angle = H$  then
8:        $kps \leftarrow \text{DetectPlayerKeypoints}(frame)$ 
9:        $Kps.add(kps)$ 
10:    else
11:      continue
12:    end if
13:  else
14:    if  $shot\_angle = O$  then
15:       $s \leftarrow \text{ShuttlecockDirPrediction}(Kps)$ 
16:       $hit \leftarrow \text{HitFrameDetection}(s)$ 
17:       $Hit.add(hit)$ 
18:       $Kps \leftarrow \emptyset$ 
19:    else
20:       $kps \leftarrow \text{DetectPlayerKeypoints}(frame)$ 
21:       $Kps.add(kps)$ 
22:    end if
23:  end if
24:   $previous\_shot\_angle \leftarrow shot\_angle$ 
25: end for
26: return  $Hit$ 

```

3. If $angle$ differs from $previous_shot_angle$ and equals O (like line 14), it indicates the end of a rally, the algorithm predicts the flying directions of the shuttlecock using keypoints stored in Kps (elaborated in) and assigns it to s . Then, it proceeds to detect hit-frames hit using `HitFrameDetection` (illustrated in) and adds it to the Hit set. The Kps set is then cleared, waiting for a new rally to begin.
4. If $angle$ differs from $previous_shot_angle$ and equals H (line 19), it indicates the start of a rally. The algorithm will process such frames like those in lines 8 and 9.

After the algorithm processes all frames in the input sequence, it returns the set Hit , which contains detected hit-frames indices of the input video.

Contribution

The main contributions of our article are to:

1. Utilize the interchange of shot angle to trim rally-wise frames from raw badminton video.
2. Proposed a novel transformer that predicts shuttlecock direction sequences based on player keypoint sequence.
3. Designed and developed the first-ever automated hit-frame detection tool to bridge the gap between raw badminton videos and analyzable data.

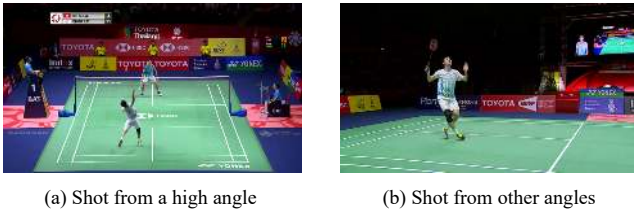


Figure 1: Frame shot from different angles.

Shuttlecock Flying Direction Prediction

This section elaborates on the proposed rally-wise video trimming, human and court keypoint detection, player detection, and shuttlecock flying direction prediction procedures. We adopted broadcast badminton videos from the public BWF YouTube channel as our data source of the dataset we utilized in the following paragraphs.

Rally-wise Video Trimming

To analyze the video of badminton matches with a 21-point scoring system simultaneously is challenging; we reduced the mission to a point-by-point (rally-by-rally) task and implemented a rally-wise video trimming module to trim all the rally-related frames from the original video. These frame segments can represent player behaviors in each rally.

We noticed that the match videos were shot from a high angle when the players competed on the court and shot from other angles while replaying highlights or filming other scenes. Figure 1 displays the example of frames that were either shot from a high angle or other angles. We formed the issue into a binary classification task and labeled each frame as either *high* (class 1) or *other* (class 0), based on its shot angle. This way, a badminton video can be seen as a long sequence of *high* and *other*. Assuming that the scenes in the rallies are always shot from a high angle, we set the frame as the rally’s start frame when the shot angle varies from *other* to *high* and the frame as the rally’s end frame when the output varies from *high* to *other*. As a result, the rally-wise video trimming module can generate rally-related frames.

Shot Angle Recognition We adopted the shot angle convolutional neural network (SA-CNN) model, as shown in Figure 2, to solve the binary classification problem. The dataset we built for the shot angle recognition task comprises 112660 frames collected from 6 unique men’s singles matches publicly available on the BWF channel. We divided them into training and testing datasets, with 62970 (4 matches) and 49690 (2 matches) frames, respectively. Additionally, badminton coaches have labeled each frame with one of the two classes, *high* and *other*. The model’s input is expected to be a three-channel RGB image x where $x \in \mathbb{R}^{3 \times 1080 \times 1920}$, and it will be normalized with the ImageNet dataset’s means and standard deviations so the pixels are scaled to a range of 0 and 1 (Deng et al. 2009). Other

preprocessing steps are defined as follows:

$$\begin{aligned} x' &= \text{Resize}(x), x' \in \mathbb{R}^{3 \times 216 \times 384} \\ x'' &= \text{CenterCrop}(x'), x'' \in \mathbb{R}^{3 \times 216 \times 216} \\ X &= \{x''_1 \cdots x''_N\}, X \in \mathbb{R}^{N \times 3 \times 216 \times 216} \\ \hat{y} &= \text{SA-CNN}(X), \hat{y} \in \mathbb{R}^{N \times C} \end{aligned}$$

Where $\text{Resize}(\cdot)$ is the function to resize the image to the specified dimensions and $\text{CenterCrop}(\cdot)$ is the function to crop the image at the center to the specified dimensions. At the same time, N and C denote the minibatch size and the number of classes, respectively. Moreover, X is the input batch, and $\hat{y}$ is the model output. The layers in SA-CNN is defined as follows:

$$\begin{aligned} H_{\text{conv}}^{(l)} &= \sum_{c=1}^{C_{\text{out}}^{(l-1)}} \sum_{i=1}^3 \sum_{j=1}^3 H_{\text{out},c,i,j}^{(l-1)} W_{c,i,j}^{(l)} + b^{(l)} \\ H_{\text{pool}}^{(l)} &= \text{MaxPool} \left(H_{\text{conv}}^{(l)}, \text{Pool}, \text{Stride} \right) \\ H_{\text{BN}}^{(l)} &= \text{BatchNorm} \left(H_{\text{pool}}^{(l)} \right) \\ H_{\text{out}}^{(l)} &= \text{ReLU} \left(H_{\text{BN}}^{(l)} \right) = \max \left(0, H_{\text{BN}}^{(l)} \right) \end{aligned}$$

where $H_{\text{conv}}^{(l)}$, $H_{\text{pool}}^{(l)}$, and $H_{\text{BN}}^{(l)}$ denotes the output of the l_{th} convolutional, maxpooling, and batch normalization layer, respectively. $C_{\text{out}}^{(l-1)}$, W , and b , represent the number of input channels, learnable weight matrices, and bias for the l_{th} convolutional layer, respectively. Pool is the pooling window size, while Stride is the pooling stride; both P and S are set as 2. The final output of the convolutional neural network is then flattened and enters a fully connected layer for classification:

$$\begin{aligned} H_{\text{flat}} &= \text{Flatten}(H_{\text{out}}^{(L)}) \\ H_{\text{FC}} &= \text{ReLU} \left(H_{\text{flat}} \cdot W_{\text{FC}} + b_{\text{FC}} \right) \\ y_{\text{pred}} &= \text{argmax}(H_{\text{FC}}), y_{\text{pred}} \in \{\text{high}, \text{other}\} \end{aligned}$$

where L stands for the layer count of SA-CNN, y_{pred} represents the predicted class, and H_{FC} represents the network’s output before applying argmax.

The SA-CNN is trained for 20 epochs with an Adam optimizer with the weight decay rate set at constant 0.1 and the learning rate set at 1e-3 but decayed 90% every six epochs. During the training phase of SA-CNN $N = 8$, $C = 2$, $L = 3$ and the loss function L that we used is shown in Eq.1.

$$L = l(\hat{y}, y) = \{l_1 \cdots l_n\}^T \quad (1)$$

$$l_n = -\log \frac{\exp(\hat{y}_{n,y_n})}{\sum_{c=1}^C \exp(\hat{y}_{n,c})} \quad (2)$$

$$l(\hat{y}, y) = \sum_{n=1}^N l_n \quad (3)$$

Court and Player Keypoints Detection

After the rally-wise video trimming module generated the frame sequence of a rally, we adopted R-CNN models to detect human and court keypoint sequences. These keypoint

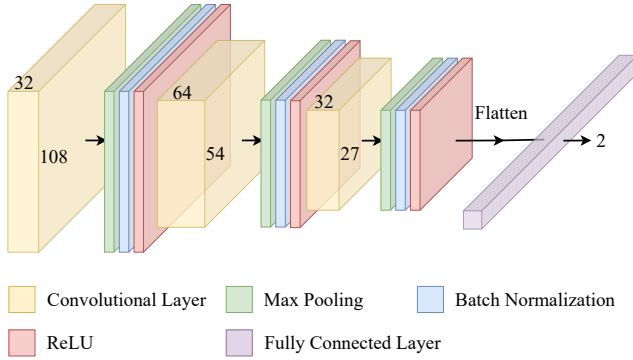


Figure 2: The SA-CNN architecture.

sequences showcase the positional and motion information of the players and will be applied in shuttlecock flying direction prediction later.

Human keypoint Detection For human keypoint detection, we adopted the Keypoint R-CNN model provided by PyTorch (Paszke et al. 2019). The Keypoint R-CNN is pre-trained with the COCO dataset (Lin et al. 2014), and its architecture is elaborated in (He et al. 2017). Mask R-CNN is a model that extends the Faster R-CNN (Girshick 2015) by adding a branch for predicting segmentation masks on each Region of Interest in parallel with the existing branch for classification and bounding box regression. The Keypoint R-CNN model is based on the Mask R-CNN method and expects its input to be an image of all sizes while the output is a list of dictionaries. Each dictionary provides the predicted box, label, confidence score, and an instance’s keypoints. In our case, we only utilized the instance’s keypoints, which include the following 17 human parts: *nose, left eye, right eye, left ear, right ear, left shoulder, right shoulder, left elbow, right elbow, left wrist, right wrist, left hip, right hip, left knee, right knee, left ankle, right ankle*. The detection process can be defined as:

$$I_{rally} \in \mathbb{R}^{1080 \times 1920}$$

$$K = \text{KeypointRCNN}(I_{rally}), K \in \mathbb{R}^{M \times 17 \times 2}$$

where I_{rally} denotes the rally-related frame, K denotes the detected human keypoint coordinates, and M represents the count of humans detected.

Court Keypoints Detection As Keypoint R-CNN inevitably returns all the humans it detected (as shown in Figure 3(a)), we trained a Court R-CNN for court keypoints detection to filter players’ information. The output format of the Court R-CNN is similar to that of the Keypoint R-CNN, except it predicted only six keypoints, including *upper right, upper left, middle right, middle left, bottom right, and bottom left* points (as shown in Figure 3(b)). Since we focus only on men’s singles matches, the detection target of the model will be the singles court.

$$K_{court} = \text{CourtRCNN}(I_{rally}), K_{court} \in \mathbb{R}^{1 \times 6 \times 2}$$

where K_{court} stands for the court’s keypoints coordinates.

Table 1: Description of KSeq dataset.

	Matches	Keypoint Sequences	Keypoint Pairs
Train	10	881	120147
Test	2	253	37288
Total	12	1134	157435

Player Keypoints Detection We detect player keypoints by integrating the two R-CNN models; the output of the Court R-CNN model is exploited to examine the ankle coordinates of the Keypoint R-CNN output instances. To determine whether the detected instance is a player, we examined its ankle coordinates and ensured it locates within the region defined by the court keypoints. Figure 3(c) displays the filtered output.

$$K_{player} = \text{Filter}(K, K_{court}), K_{player} \in \mathbb{R}^{2 \times 17 \times 2},$$

where K_{player} stands for the players’ keypoints and $\text{Filter}(\cdot)$ is the function that takes the court and human’s keypoints as its input and outputs the on-court player’s keypoints.

Shuttlecock Flying Direction Prediction

Identifying the shuttlecock’s flying direction is essential for downstream analysis since it implies information about when the players hit the shuttlecock. In a badminton rally, shots will occur several times until it ends, when a fault happens, or when the shuttlecock hits the floor. Therefore, we conclude the flying status of the shuttlecock into the following three state classes: *steady, flying toward the bottom court, and flying toward the upper court*, then formulate the shuttlecock flying direction prediction as a sequence labeling task with tokens S , B , and U representing the three classes respectively. We assume the player keypoint sequence is essential in predicting the shuttlecock’s flying direction. Consequently, we proposed a novel transformer model that leverages keypoint sequences to predict shuttlecock flying direction sequences.

For the shuttlecock flying direction prediction task, we built a keypoint sequence dataset (KSeq dataset) with 12 unique men’s singles matches from the BWF channel as the data source. Ten matches were used to train the transformer model, while two were for testing. The sequence data in the KSeq dataset comprises multiple frame-wise keypoint pairs (2 players). Table 1 shows the information of KSeq dataset. In total, the dataset is composed of 1134 rally-wise keypoint sequences. For the ground truth, we invited several badminton experts to label each keypoint pair with one of the mentioned three classes according to the shuttlecock flying direction in the corresponding video frame. Furthermore, padding token (token Pad) is applied to ensure each keypoint sequence has the same length during training, and the keypoints will be normalized with the dataset’s average and standard deviation before entering the transformer.

We trained the transformer for 100 epochs with an Adam optimizer, whose learning rate is set as $1e-5$ and decayed 90% after the seventieth epoch. The training process is de-



Figure 3: (a) Illustrates the Keypoint R-CNN model's output without filtering the instances. (b) shows the output of the Court R-CNN model. (c) Illustrates the outcome of the integrated method of Keypoint R-CNN and Court R-CNN.

defined as follows:

$$\begin{aligned} x^t &= (K_{player(1)} \cdots K_{player(F)}) \\ X^t &= (x_1^t \cdots x_N^t), x^t \in \mathbb{R}^{F \times 2 \times 17 \times 2} \\ \hat{y}^t &= \text{Transformer}(X^t), \hat{y}^t \in \mathbb{R}^{N \times F \times C} \end{aligned}$$

where x^t stands for the rally-wise keypoint sequences, X^t represents the training data batch, y^t is the shuttlecock direction sequence, and F represents the frame sequence length. The loss function L^t for transformer training is defined as follows:

$$y^t = \{y_1^t \cdots y_N^t\}, y_n^t \in \{S, B, U, Pad\} \quad (4)$$

$$L^t = l^t(\hat{y}^t, y^t) = \{l_1^t \cdots l_N^t\}^T \quad (5)$$

$$Ignore(x) = \begin{cases} 1, & \text{if } x \neq \text{ignore index} \\ 0, & \text{else} \end{cases} \quad (6)$$

$$l_n^t = -\log \left(\frac{\exp(\hat{y}_{n,y_n}^t)}{\sum_{c=1}^C \exp(\hat{y}_{n,c}^t)} \right) \cdot Ignore(y_n^t) \quad (7)$$

$$l^t(\hat{y}^t, y^t) = \sum_{n=1}^N \frac{l_n^t}{\sum_{n=1}^N Ignore(y_n^t)} \quad (8)$$

In the loss function L^t , N spans the batch dimension as well as F . During training, $N = 1, F = 600, C = 4$, and the padding token Pad is set as the ignore index, so it is ignored and does not contribute to the input gradient when calculating the loss. Figure 4 depicts the transformer's structure.

Player-Wise Projection A player-wise projection layer is built to initially project the input keypoints to a higher dimension before feeding the transformer sequences of numerous pairs of keypoints. The player-wise projection layer comprises two multi-layer perceptron blocks with two fully connected layers. Before entering the layer, the keypoint data of the two players will be momentarily separated, and following the projection procedure, they will be concatenated. In our experiment, the transformer model with player-wise projection layers outperforms the model without it, supporting our hypothesis that this mechanism helps separate the information of the two players rather than considering them as a single identity. The player-wise projection

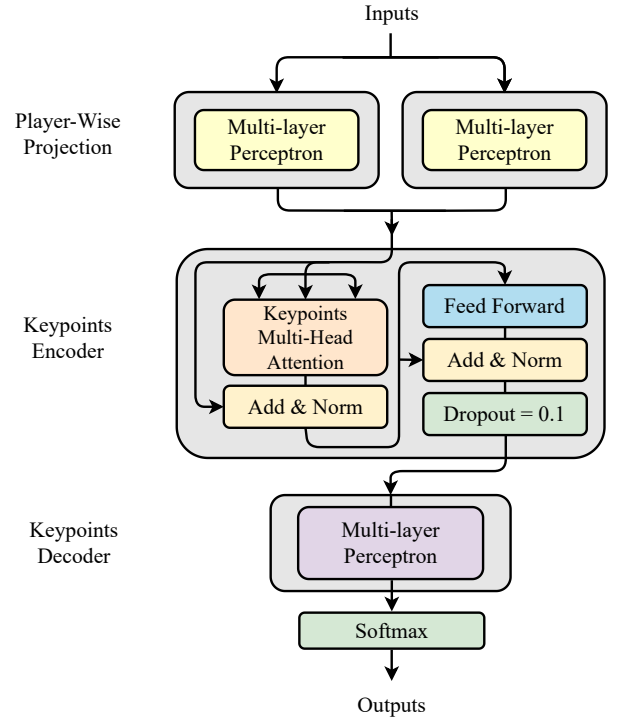


Figure 4: The architecture of the adopted transformer model.

layer is defined as follows:

$$\begin{aligned} FC(x) &= \text{ReLU}(W_{FC} \cdot x + b_{FC}) \\ H_{\text{player}}^{(i)} &= \text{FC}(\text{FC}(x^t[:, i, :, :])) , i \in \{1, 2\} \\ x_{\text{projected}}^t &= \text{Concatenate}(H_{\text{player}}^{(1)}, H_{\text{player}}^{(2)}) \end{aligned}$$

where W_{FC}, b_{FC} is the learnable weight matrices and bias of the fully connected layer, $H_{\text{player}}^{(i)}$ denotes the output of the i_{th} player-wise projection layer before concatenation, and $x_{\text{projected}}^t$ represents the projected keypoint sequence tensor.

keypoint Encoder and Decoder The encoder stack comprises several identical layers, each built based on the architecture described in (Vaswani et al. 2017) and consists

of two sub-layers. The first is a multi-head self-attention layer; the second is a position-wise fully connected feed-forward network. Around the two sub-layers, residual connection is proposed, followed by layer normalization and dropout. The model’s outputs from all sub-layers have identical dimensions to facilitate these residual connections. The transformer model we built features eight encoder layers and eight heads for the attention mechanism; the dimension of the feed-forward layer is 2048.

Hit-Frame Detection

Hit-frame, which are defined as the frame that a player hits the shuttlecock and thus changes its flying direction, can be utilized to retrieve data for downstream analysis since they possess information about the players when they hit the shuttlecock. Such hit-frames can be identified by detecting token shifts in the shuttlecock flying direction sequence. Algorithm 2 depicts how we devised hit-frame detection.

Lines 1 to 3, the algorithm starts by defining three constants: S , B , and U , which represent the states of the shuttlecock flying direction as follows: *steady*, *flying toward the bottom court*, and *flying toward the upper court*. The variable $previous_direction$ is initialized to S to keep track of the previous flying direction encountered during the iteration. The set hit is also created and initialized as empty, where the identified hit-frame indices will be stored. The algorithm then enters a loop that iterates over each shuttlecock flying direction in the input sequence s . If the current direction differs from the previous direction within the loop, the shuttlecock’s flying direction has changed. The algorithm then checks for specific cases of transitions to detect hit-frames.

1. If the previous direction was S and the current direction is B or U , it tells that a player starts the rally; therefore, the algorithm identifies this frame as a hit-frame. It calls `FrameIndex` to get the frame index h corresponding to the current direction and adds it to the set hit .
2. If the previous direction was B and the current direction is U , or vice versa, it indicates the player at the bottom court strikes the shuttlecock, causing a flying direction transition (or vice versa). The algorithm calls `FrameIndex` to obtain the frame index h corresponding to the current direction and adds it to the set hit .

The algorithm then updates the $previous_direction$ variable to the current direction before moving on to the next iteration. After all directions in the sequence have been processed, the algorithm returns the set hit containing the detected hit-frame indices, representing potential badminton shots where the shuttlecock changes its flying direction.

Experiments

Our research intends to enhance sports analysis in badminton by systematically detecting hit-frames from match videos. We seek to address three key research questions:

1. Can rallies be precisely extracted from match videos?
2. Can shuttlecock flying direction sequence be transformed from the sequence of keypoints of players in frames?

Algorithm 2: Hit-Frame detecting algorithm.

Input: Shuttlecock flying direction sequence s .

Output: Hit-frames set hit .

```

1: Let const  $S \leftarrow 0, B \leftarrow 1, U \leftarrow 2$ 
2: Let  $previous\_direction \leftarrow S$ 
3: Let  $hit \leftarrow \emptyset$ 
4: for all  $direction \in s$  do
5:   if  $previous\_direction \neq direction$  then
6:     if  $previous\_direction = S$  then
7:        $h \leftarrow \text{FrameIndex}(direction)$ 
8:        $hit.add(h)$ 
9:     else if  $previous\_direction = B$  then
10:      if  $direction = U$  then
11:         $h \leftarrow \text{FrameIndex}(direction)$ 
12:         $hit.add(h)$ 
13:      end if
14:    else if  $previous\_direction = U$  then
15:      if  $direction = B$  then
16:         $h \leftarrow \text{FrameIndex}(direction)$ 
17:         $hit.add(h)$ 
18:      end if
19:    end if
20:  end if
21:   $previous\_direction \leftarrow direction$ 
22: end for
23: return  $hit$ .
```

Table 2: Confusion matrix of the SA-CNN.

Accuracy	Precision	Recall	F1-Score
0.9976	0.9903	1.0	0.9951

3. Can hit-frames be precisely identified through flying direction changes?

We will answer these questions in the following paragraphs, and the assessment and experiment of the SA-CNN, rally-wise video trimming module, transformer, and hit-frame detecting module are the main topics of this section.

SA-CNN

We adopted the testing dataset mentioned in to evaluate SA-CNN. Such dataset comprising of two matches (49690 frames), and all of its frames are labeled with one of the two classes, *high* and *other*. The confusion matrix for the evaluation is displayed in Table 2. We can observe that SA-CNN has remarkable performance and great accuracy for identifying shot angles.

Rally-Wise Video Trimming

While the frames were inputted sequentially into SA-CNN, a continuous sequence of *high* and *other* will be outputted. In this study, we evaluated our rally-wise video trimming module with S^2 -Labeling, a shot-by-shot labeling dataset that consists of six badminton matches (Huang et al. 2022); however, we could only find four available video sources of the dataset so those accessible videos will be used for evaluation. The S^2 -Labeling dataset provides the rally’s start

Table 3: Numbers of correctly trimmed, extra trimmed, miss trimmed videos and the actual rally counts.

Correct	Extra	Missed	Total Trimmed	Actual
287	33	36	320	323

Table 4: Confusion matrix of the rally-wise video trimming module.

Accuracy	Precision	Recall	F1-Score
0.8062	0.8969	0.8885	0.8927

and end frame number, which we adopted in the evaluation stage. Table 3 shows the number of rallies trimmed and the actual rally count, while Table 4 shows the confusion matrix of the SA-CNN. From Tables 3 and 4, we acknowledged that the video trimming module could not trim all rally-related videos. Such a limitation is due to the varying shooting forms of broadcast badminton videos. This restriction is inevitable as the broadcasting department aims to enhance the audience’s viewing experience. However, most of the time, the rally-wise video trimming module can still trim rally-related frames from raw match videos and achieve high precision and recall value.

Transformer

With the transformer model, player keypoint sequences can be transformed into shuttlecock flying direction sequences comprising *steady* (token S), *flying toward the bottom court* (token B), and *flying toward the upper court* (token U). Therefore, for transformer evaluation, we will calculate the confusion matrix of the predicted tokens with the KSeq testing dataset. The evaluation result is shown in Table 5. Table 5 shows that the transformer model performs better when predicting token S . Overall, the transformer can robustly tackle the shuttlecock’s flying direction prediction task.

Hit-Frame Detection

Since we have trained the transformer model with only men’s singles matches, for the evaluation of the hit-frame detection module, we exploited two men’s singles matches from the S²-Labeling dataset (the other two matches are women’s singles matches) and the KSeq testing dataset in the evaluation stage; plus, the frames per second (FPS) of match videos are all 30. We assess the hit-frame detection module within three conditions: ± 5 , ± 15 , ± 25 , represent the range of the difference between the predicted hit-frame’s number and the actual hit-frame’s number. If the difference is within the range, the prediction will be considered correct.

Table 5: Confusion matrix of the transformer model.

Token	Accuracy	Precision	Recall	F1-Score
S	0.9717	0.9798	0.9410	0.9600
B	0.9275	0.8879	0.8976	0.8927
U	0.9295	0.8824	0.9090	0.8955

Table 6: Confusion matrix of the hit-frame detecting module.

Condition	Accuracy	Precision	Recall	F1-Score
± 5	0.9585	0.5454	0.7721	0.6392
± 15	0.9765	0.6787	0.9608	0.7955
± 25	0.9782	0.6915	0.9790	0.8105

For instance, condition ± 15 is defined as:

$$\forall n \in \mathbb{Z}, \text{ if } n \in n_{\text{hit}}, P \text{ is correct if } P \in (n - 15, n + 15)$$

where n represents the frame number, n_{hit} denotes the set of actual hit-frame numbers, and P is the prediction. The result of hit-frame detection is shown in Table 6. From Table 6, we can infer that the hit-frame detecting module can distinguish most hit-frames within the range ± 15 .

Hit-Frame Applications

Numerous current studies on badminton involve hit-frames. For example, the badminton observational tool validated by (Valdecabres et al. 2020) defined on-court movement as a movement done by the observed player, considering the starting and ending zones. The starting zone refers to the quadrant where the observed player is when the opponent hits the shuttlecock, and the ending zone refers to the quadrant where the observed player hits the shuttle back to the opponent’s court. Moreover, (Chu and Situmeang 2017) classify the stroke a player invokes based on his/her pose when he/she hits the shuttlecock. (Wang et al. 2021) introduce a badminton language BSLR to describe the process of the shot. The BSLR data format includes an essential attribute: timestamp, which is the shot’s hitting time (in seconds). Our hit-frame detection framework can obtain such an attribute; therefore, downstream applications based on BSLR like (Wang 2022) and (Wang et al. 2022a), which captures a shot-by-shot sequence in a badminton rally, can be realized through our framework.

Conclusion

Badminton analysis has predominantly relied on manual methods or coach records. Therefore, scalable, automated information extraction is crucial for the training of AI models. In this paper, we developed an automated information extraction model that captures rally-by-rally information from publicly available badminton videos, bridging the gap between videos and analyzable information. We trimmed the rally frame sequences from raw badminton match videos based on the interchange of shot angle, detected hit-frames, which possess abundant player information, with the shifts in shuttlecock flying direction, and achieved 81%, 96% accuracy, respectively. Our approach can further support AI-based applications like stroke classification, movement recognition, and tactical analysis. The badminton community can more effectively assess badminton matches from hundreds of publicly available BWF films with the aid of the hit-frame detection framework. Our code and datasets will be made available to the general public.

References

- Abdullahi, Y.; and Coetzee, B. 2017. Notational singles match analysis of male badminton players who participated in the African Badminton Championships. *International Journal of Performance Analysis in Sport*, 17(1-2): 1–16.
- Careelmont, S. 2013. Badminton shot classification in compressed video with baseline angled camera. *Master esis, University of Ghent*.
- Chen, B.; and Wang, Z. 2007. A statistical method for analysis of technical data of a badminton match based on 2-D se-riate images. *Tsinghua science and technology*, 12(5): 594–601.
- Chu, W.-T.; and Situmeang, S. 2017. Badminton video anal-ysis based on spatiotemporal and stroke features. In *Pro-ceedings of the 2017 ACM on international conference on multimedia retrieval*, 448–451.
- Deng, J.; Dong, W.; Socher, R.; Li, L.-J.; Li, K.; and Fei-Fei, L. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, 248–255. Ieee.
- Ghosh, A.; Singh, S.; and Jawahar, C. 2018. Towards struc-tured analysis of broadcast badminton videos. In *2018 IEEE Winter Conference on Applications of Computer Vi-sion (WACV)*, 296–304. IEEE.
- Girshick, R. 2015. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, 1440–1448.
- He, K.; Gkioxari, G.; Dollár, P.; and Girshick, R. 2017. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, 2961–2969.
- Huang, K.-S.; Huang, C.; Chung, S. S.; and Tsai, C.-L. 2016. Kinematic analysis of three different badminton backhand overhead strokes. In *ISBS-Conference Proceedings Archive*.
- Huang, Y.-H.; Huang, Y.-C.; Lee, H. S.; İk, T.-U.; and Wang, C.-C. 2022. S 2-Labeling: Shot-By-Shot Microscopic Badminton Singles Tactical Dataset. In *2022 23rd Asia-Pacific Network Operations and Management Symposium (APNOMS)*, 1–6. IEEE.
- Kuntze, G.; Mansfield, N.; and Sellers, W. 2010. A biomechanical analysis of common lunge tasks in badminton. *Journal of sports sciences*, 28(2): 183–191.
- Lee, C.-L. 2016. Badminton Shuttlecock Tracking and 3D Trajectory Estimation From Video.
- Lin, T.-Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ra-manan, D.; Dollár, P.; and Zitnick, C. L. 2014. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzer-land, September 6-12, 2014, Proceedings, Part V 13*, 740–755. Springer.
- Liu, P.; and Wang, J.-H. 2022. MonoTrack: Shuttle trajec-tory reconstruction from monocular badminton video. In *Proceedings of the IEEE/CVF Conference on Computer Vi-sion and Pattern Recognition*, 3513–3522.
- Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information pro-cessing systems*, 32.
- Phomsoupha, M.; and Laffaye, G. 2015. The science of badminton: game characteristics, anthropometry, physiolo-gy, visual fitness and biomechanics. *Sports medicine*, 45: 473–495.
- Ramasamy, Y.; Usman, J.; Sundar, V.; Towler, H.; and King, M. 2021. Kinetic and kinematic determinants of shuttlecock speed in the forehand jump smash performed by elite male Malaysian badminton players. *Sports biomechanics*, 1–16.
- Rambely, A. S.; and Osman, N. A. A. 2005. The contribution of upper limb joints in the development of racket velocity in the badminton smash. In *ISBS-Conference Proceedings Archive*.
- Sun, N.-E.; Lin, Y.-C.; Chuang, S.-P.; Hsu, T.-H.; Yu, D.-R.; Chung, H.-Y.; and İk, T.-U. 2020. TrackNetV2: Efficient shuttlecock tracking network. In *2020 International Con-ference on Pervasive Artificial Intelligence (ICPAI)*, 86–91. IEEE.
- Valldcabres, R.; Casal, C. A.; Chiminazzo, J. G. C.; and De Benito, A. M. 2020. Players’ on-court movements and contextual variables in badminton world championship. *Frontiers in Psychology*, 11: 1567.
- Valldcabres, R.; de Benito, A.; Casal, C.; and Pablos, C. 2019. DESIGN AND VALIDITY OF A BADMINTON OBSERVATION TOOL (BOT). *Revista internacional de medicina y ciencias de la actividad fisica y del deporte*, 19(74).
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. At-tention is all you need. *Advances in neural information pro-cessing systems*, 30.
- Wang, W.-Y. 2022. Modeling Turn-Based Sequences for Player Tactic Applications in Badminton Matches. In *Pro-ceedings of the 31st ACM International Conference on In-formation & Knowledge Management*, 5128–5131.
- Wang, W.-Y.; Chan, T.-F.; Peng, W.-C.; Yang, H.-K.; Wang, C.-C.; and Fan, Y.-C. 2022a. How Is the Stroke? Inferring Shot Influence in Badminton Matches via Long Short-Term Dependencies. *ACM Transactions on Intelligent Systems and Technology*, 14(1): 1–22.
- Wang, W.-Y.; Chan, T.-F.; Yang, H.-K.; Wang, C.-C.; Fan, Y.-C.; and Peng, W.-C. 2021. Exploring the long short-term dependencies to infer shot influence in badminton matches. In *2021 IEEE International Conference on Data Mining (ICDM)*, 1397–1402. IEEE.
- Wang, W.-Y.; Shuai, H.-H.; Chang, K.-S.; and Peng, W.-C. 2022b. Shuttlenet: Position-aware fusion of rally progress and player styles for stroke forecasting in badminton. In *Proceedings of the AAAI Conference on Artificial Intelli-gence*, volume 36, 4219–4227.
- Wang, Z.; Guo, M.; and Zhao, C. 2016. Badminton stroke recognition based on body sensor networks. *IEEE Transac-tions on Human-Machine Systems*, 46(5): 769–775.